昨年度作品からの成長

～『Slime's Sky Travel』から『Slime's Space Travel』へ～

2026. 11. 5
日下 拓海

1.　本資料について

昨年度の応募作品『Slime's Sky Travel』と、本年度の『Slime's Space Travel』を比較し、この 1 年間の技術力・設計力・ゲーム開発に対する考え方の成長をまとめました。前作で見つかった課題を本年度作品でどう改善し、遊びやすさや分かりやすさにつなげたかを中心に紹介します。

1-1.　昨年度から本年度への主な成長

分野	昨年度作品	本年度作品
描画・3D 表現	固定機能中心の OpenGL・プリミティブ表現	GLSL Shader・3D モデル・スケルタルアニメーション
移動・物理処理	Y 軸固定の重力・AABB 中心の衝突判定	球・楕円体の 360°移動・Ray Test・Sweep Test
開発環境・設計	設定値の編集・ホットリロード	画面上での直接編集・機能ごとの責務分割
遊びやすさ	移動・ジャンプ中心・Easy Mode	戦闘・らくらくスタイル・2 人プレイ・自動ルビ
プレイヤー向け機能	ステージのプレイが中心	ステージ作成・保存・プレイテスト

本年度は、技術基盤を発展させながら戦闘や 2 人プレイなど遊びの幅を広げました。その一方で、機能の増加によって操作や理解の負担まで増えないようにし、**「遊びの幅を広げても、遊ぶハードルは上げない」**ことを一つの軸として開発しました。

昨年度作品



本年度作品



昨年度作品と本年度作品のゲーム画面

2. 描画・3D 表現の成長

2-1. 固定機能パイプラインからプログラマブルパイプラインへ

昨年度作品では、glBegin / glVertex3f / glMatrixMode などを用いた OpenGL の固定機能・即時モードを中心に描画し、プレイヤーや足場もプリミティブへテクスチャを貼って表現していました。

本年度作品では、前作での経験を踏まえ、VAO / VBO / EBO と GLSL Shader を用いたプログラマブルパイプラインによる描画基盤を新たに構築しました。頂点データや描画処理を自分で管理し、3D モデル、アニメーション、Shadow Map などを扱える構成へ発展させました。

2-2. 3D モデル・スケルタルアニメーションの導入

昨年度作品では、キャラクターの状態や行動をアニメーションで表現する仕組みがありませんでした。

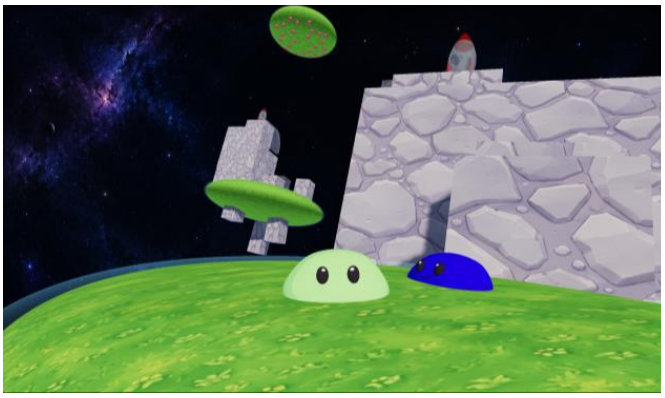
本年度作品では、Assimp を用いて FBX モデル、Bone Weight、Animation Clip を読み込み、キーフレーム補間と Quaternion SLERP によるスケルタルアニメーションを実装しました。これにより、待機、移動、攻撃、回避などをキャラクター自身の動きとして表現できるようにしました。

また、Shadow Map によって接地位置や高低差を伝えやすくし、Post Process によって攻撃や演出の視覚的なフィードバックを強めることを目指しました。技術の導入だけでなく、プレイヤーが自分の行動やゲームの状況を理解しやすい表現も意識しています。

昨年度作品



本年度作品



プリミティブ表現から 3D モデルを用いた表現へ

3. 移動・物理処理の成長

3-1. Y 軸固定の移動から球・楕円体の 360°移動へ

昨年度作品では、重力方向をワールド座標の-Y 方向に固定し、Y 軸を常に上として扱っていました。

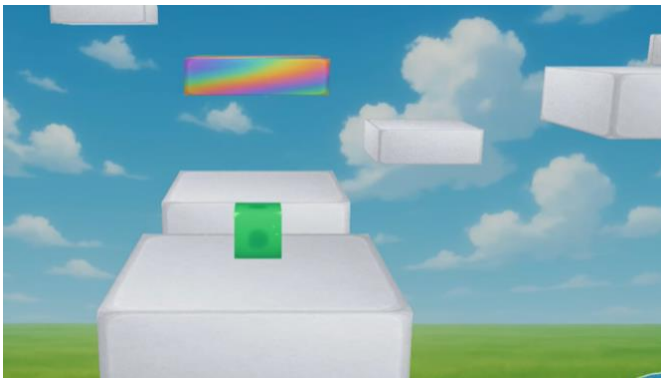
本年度作品では惑星の表・側面・裏面までを一つのステージとして利用するため、位置に応じて上方向や重力方向を変化させる必要がありました。

そこで Actor ごとに上方向を表す upVec を持たせ、接地時には Ray Test で取得した地面法線を基準に姿勢と重力方向を更新する仕組みを実装しました。また、空中では引力対象の惑星に応じて上方向を変化させています。

これにより、惑星全体を探索できる 360°アクションを実現しながら、姿勢や重力方向が変わっても自然に操作できることを目指しました。

昨年度作品

本年度作品




Y 軸固定のステージから球・楕円体を利用した 360°移動へ

3-2. AABB 中心の判定から Ray Test・Sweep Test へ

昨年度作品の衝突判定は、プレイヤーと足場の最小・最大座標を比較する AABB 判定が中心でした。

本年度作品では複雑な地形を扱うため、Bullet Physics を利用した接地・衝突判定へ発展させました。接地判定では中央・前後左右から Ray Test を行い、取得した地面法線から upVec を決定しています。

また、移動時には Sweep Test によって移動経路上の衝突を検出し、衝突時には残りの移動量から壁の法線方向へ入り込む成分を除いて、壁沿いに移動させています。これにより、360°移動に対応しながら、壁への引っ掛かりやすり抜けを抑える仕組みを実装しました。

4. 開発環境・設計の成長

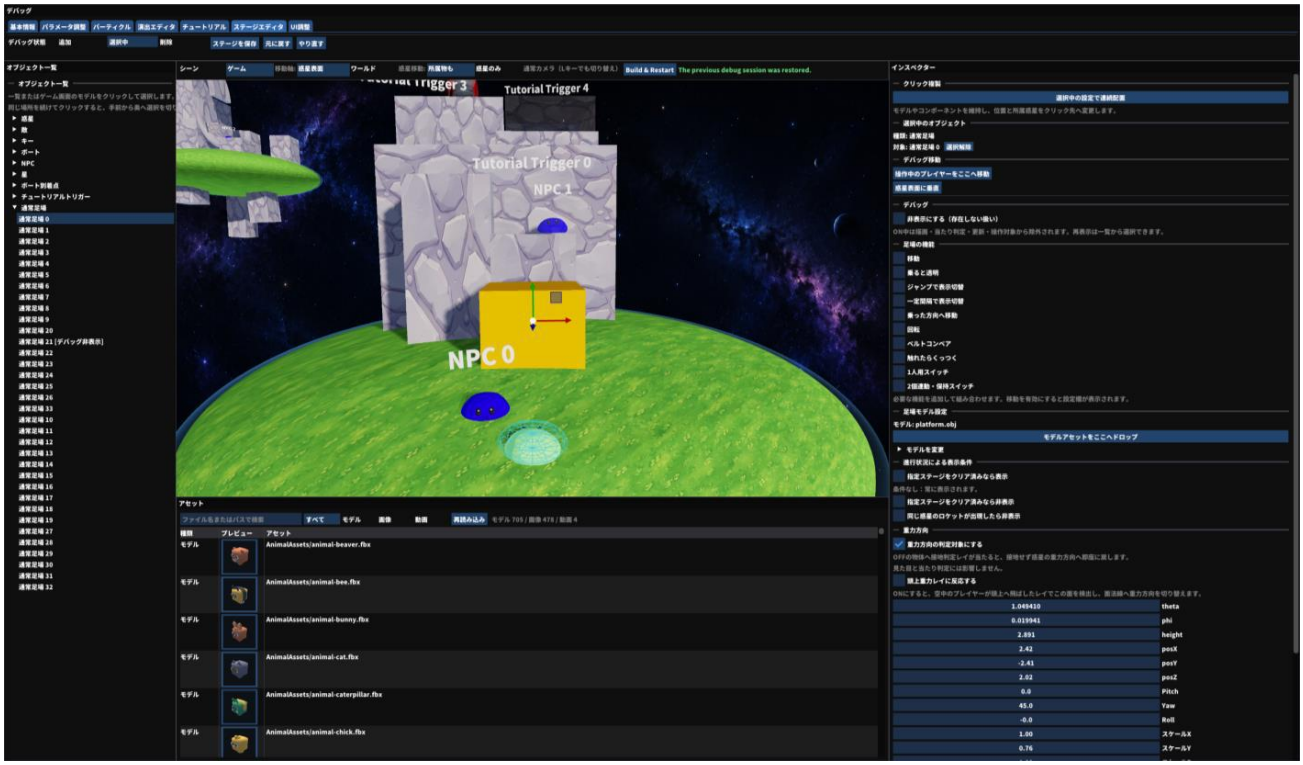
4-1. 数値編集・ホットリロードから画面上での直接編集へ

昨年度作品でも、Stage・Texture・Audio・UI Config を実行中に再読み込みするホットリロードを用意していました。しかし、足場や UI の配置調整では、設定値を編集してからゲーム画面で結果を確認する往復が必要でした。

本年度作品では、ゲーム画面を Framebuffer へ描画して Debug Editor 内に表示し、ImGuizmo によって Actor を直接移動・回転・拡大縮小できる 3D ステージエディタを実装しました。また、UI も実際のゲーム画面上で選択し、配置変更・複製・削除・Undo・保存を行えるようにしました。

さらに、C++コード変更時に Editor の作業状態を保存し、ビルド・再起動後に復元する Build & Restart も実装しました。これにより、コード変更後も元の編集状態から作業を再開できるようにしました。

画面を確認しながらステージや UI を調整でき、繰り返し改善しやすい制作環境へ発展させました。



ゲーム画面上で Actor を直接編集できる 3D ステージエディタ

4-2. 機能追加に耐えられる責務分割

昨年度作品では、GameState が Player、Camera、Item、Skill、Replay、UI など多くの状態を保持し、Platform にも複数の役割が集まっていました。

本年度作品では、Player の処理を PlayerInput、PlayerMovement、PlayerGrounding、PlayerCombat などへ分割しました。Enemy についても移動・戦闘・状態管理・行動 AI を分離し、ゲーム全体も System や Controller へ責務を分けています。

クラスを細かくすること自体ではなく、操作感なら Movement、戦闘なら Combat というように変更箇所と影響範囲を判断しやすくすることを重視しました。機能を増やしながらも、各要素を独立して調整・改善できる設計を目指しています。

5. 遊びの幅と遊びやすさの両立

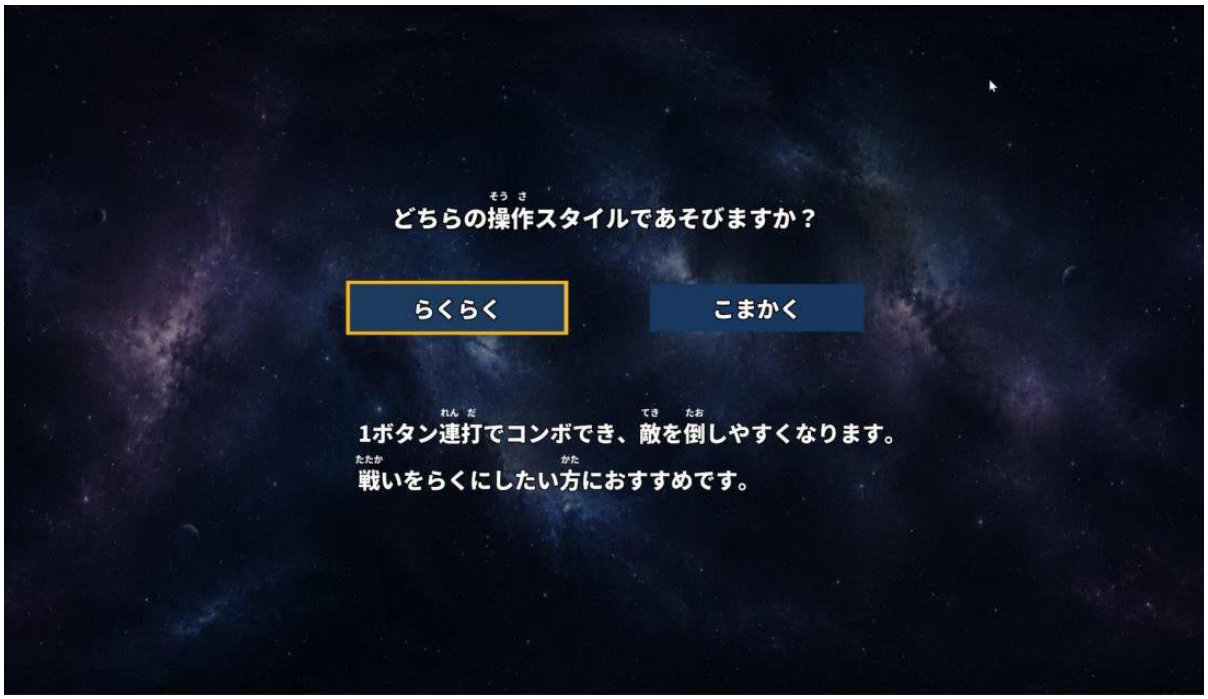
5-1. 戦闘追加と「らくらく／こまかく」スタイル

昨年度作品は移動とジャンプを中心とした構成でしたが、Easy Mode によって落下時の救済や制限時間の延長など、アクションが苦手な人でも進めやすくする工夫を行っていました。

本年度作品では、コンボ・強攻撃・ガード破壊・打ち上げ・空中攻撃などを組み合わせられる戦闘を追加しました。一方で、選べる行動が増えるほど操作も複雑になります。

そこで、技を自分で使い分ける「こまかくスタイル」に加え、攻撃対象の選択や自動方向転換、敵の状態に応じた攻撃・空中追撃の補助を行う「らくらくスタイル」を実装しました。これにより、同じ攻撃ボタンの連打でも多彩なアクションを楽しめるようにしました。

前作の**「失敗したときの負担を減らす支援」**から、**「複雑な遊びそのものへ参加しやすくする支援」**へと考え方を発展させています。



プレイヤーに合わせて操作方法を選べる「らくらく／こまかく」スタイル

5-2. プレイ人数・入力方法・文字表示への対応

昨年度作品はキーボード操作を中心とした 1 人用の構成でした。本年度作品ではゲームパッドとローカル 2 人プレイに対応し、1 人では分身した 2 体のスライムを切り替え、2 人ではそれぞれが 1 体ずつ操作できるようにしました。

また、昨年度の英語中心の Bitmap Font による表示から、SDL_ttf による日本語描画へ変更しました。Windows 版では JapanesePhoneticAnalyzer を利用し、漢字への自動ルビ表示にも対応しています。

遊び方の選択肢を広げるだけでなく、一緒に遊ぶ人の有無や文字を読む力によって、ゲームを楽しみにくくなる状況を減らすことを目指しました。

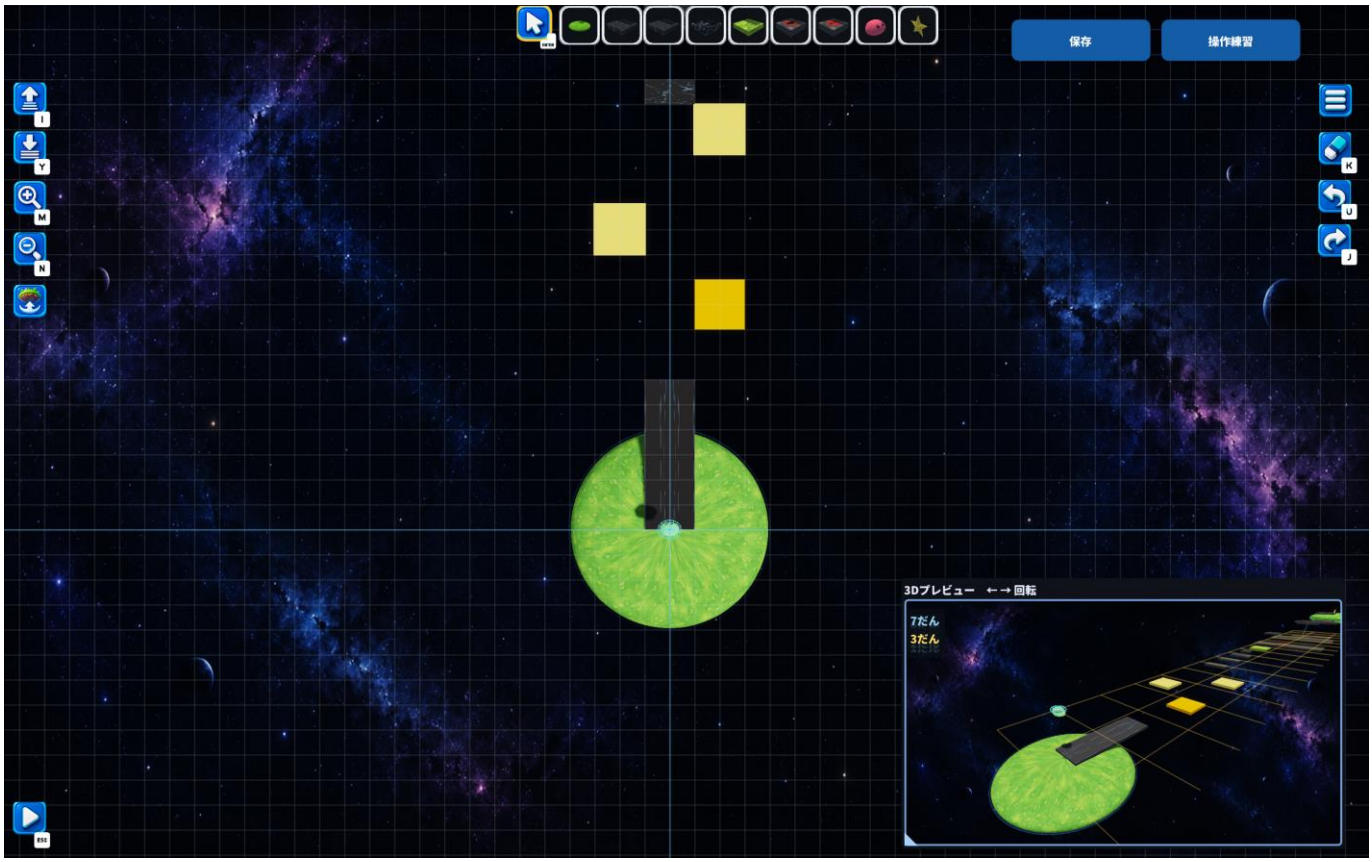
6. プレイヤー向けステージ作成機能

6-1. 「遊ぶ人」から「遊びを作る人」へ

昨年度作品には、プレイヤー自身がステージを作成する機能はありませんでした。本年度作品では、ゲームを遊ぶだけでなく自分でステージを作って楽しめるよう、プレイヤー向けのステージ作成機能を実装しました。

一般的な 3D エディタのように X・Y・Z 軸を自由に操作する方式では、3D 編集に慣れていない人が位置関係を把握しにくいと考えました。そこで、水平方向を 2D グリッド、高さ方向を「だん」として分離し、マス単位の配置と高さの切り替えで立体的なステージを作成できる仕組みにしました。

また、3D プレビュー・Undo / Redo・保存・プレイテスト・クリア確認まで一連の流れを用意しました。内部では 3D 座標を扱いながら、プレイヤーには専門的な 3D 操作をできるだけ意識させず、積み木を組み立てるような感覚でステージを制作できることを目指しました。



2D 操作と 3D プレビューを組み合わせたプレイヤー向けステージ作成機能

7. まとめ

この 1 年間で、固定機能中心の描画から Shader・3D モデル・アニメーションへ、Y 軸固定の移動から球・楕円体上の 360°移動へ、数値編集からゲーム画面上での直接編集へと、ゲーム開発を支える技術基盤を発展させました。

また、責務分割や開発用エディタの整備を通して、機能を実装するだけでなく、その後も継続的に調整・改善しやすい設計の重要性を学びました。さらに、戦闘・2 人プレイ・自動ルビ・ステージ作成機能などを追加する際には、遊びの幅を広げながら、初心者や子どもでも楽しめることを重視しました。

今後も、**遊びの深さを保ちながら、それを楽しめる人を技術によって広げられるゲーム開発**を目指します。